

数値計算 逆行列プログラム

3S-29 辻本純平

1. ソースコード
(ア) inverse_matrix.c

```
/******  
  
    matinv.c -- 逆行列  
******/  
  
#include "matutil.c" /* 行列操作小道具集 */  
#include <math.h>  
  
double lu(int n, matrix a, int *ip) /* LU 分解 */  
{  
    int i, j, k, ii, ik;  
    double t, u, det;  
    vector weight;  
  
    weight = new_vector(n); /* weight[0..n-1] の記憶領域確保 */  
    det = 0; /* 行列式 */  
    for (k = 0; k < n; k++) { /* 各行について */  
        ip[k] = k; /* 行交換情報の初期値 */  
        u = 0; /* その行の絶対値最大の要素を求める */  
        for (j = 0; j < n; j++) {  
            t = fabs(a[k][j]); if (t > u) u = t;  
        }  
        if (u == 0) goto EXIT; /* 0 なら行列は LU 分解できない */  
        weight[k] = 1 / u; /* 最大絶対値の逆数 */  
    }  
    det = 1; /* 行列式の初期値 */  
    for (k = 0; k < n; k++) { /* 各行について */  
        u = -1;  
        for (i = k; i < n; i++) { /* より下の各行について */
```

```

        ii = ip[i];          /* 重み×絶対値 が最大の行を見つける */
        t = fabs(a[ii][k]) * weight[ii];
        if (t > u) {    u = t;    j = i;    }
    }
    ik = ip[j];
    if (j != k) {
        ip[j] = ip[k];    ip[k] = ik;    /* 行番号を交換 */
        det = -det;    /* 行を交換すれば行列式の符号が変わる */
    }
    u = a[ik][k];    det *= u;    /* 対角成分 */
    if (u == 0) goto EXIT;    /* 0 なら行列は LU 分解できない */
    for (i = k + 1; i < n; i++) {    /* Gauss 消去法 */
        ii = ip[i];
        t = (a[ii][k] / u);
        for (j = k + 1; j < n; j++)
            a[ii][j] -= t * a[ik][j];
    }
}
EXIT:
    free_vector(weight);    /* 記憶領域を解放 */
    return det;            /* 戻り値は行列式 */
}

```

```
double matinv(int n, matrix a, matrix a_inv)
```

```

{
    int i, j, k, ii;
    double t, det;
    int *ip;    /* 行交換の情報 */

    ip = malloc(sizeof(int) * n);
    if (ip == NULL) error("記憶領域不足");
    det = lu(n, a, ip);
    if (det != 0)
        for (k = 0; k < n; k++) {
            for (i = 0; i < n; i++) {
                ii = ip[i];    t = (ii == k);

```

```

        for (j = 0; j < i; j++)
            t -= a[i][j] * a_inv[j][k];

        a_inv[i][k] = t;
    }

    for (i = n - 1; i >= 0; i--) {
        t = a_inv[i][k];  ii = ip[i];
        for (j = i + 1; j < n; j++)
            t -= a[i][j] * a_inv[j][k];

        a_inv[i][k] = t / a[i][i];
    }
}

free(ip);
return det;
}

/***** 以下はテスト用 *****/

#include <limits.h>

double rnd(void) /* 乱数 0 < rnd() < 1 */
{
    static unsigned long seed = 123456789UL; /* 奇数 */

    return (seed *= 69069UL) / (ULONG_MAX + 1.0);
}

int main(void)
{
    int i, j, n;

    matrix a, gyaku, teisu, x, b_check; // 変数を宣言

    printf("行列の次元 n = ");
    scanf("%d", &n);

    printf("係数行列 A (n x n) を入力してください:¥n");
    a = new_matrix(n, n);

```

```

for (i = 0; i < n; i++){
    for (j = 0; j < n; j++){
        printf("a[%d][%d] = ", i, j);
        scanf("%lf", &a[i][j]);
    }
}

// 定数行列は「n 行 1 列」の列ベクトルとして扱うのが一般的です
printf("定数行列 b (n x 1) を入力してください:¥n");
teisu = new_matrix(n, 1);
for (i = 0; i < n; i++) {
    printf("b[%d][0] = ", i);
    scanf("%lf", &teisu[i][0]);
}

// --- 逆行列の計算 ---
gyaku = new_matrix(n, n);
double det = matinv(n, a, gyaku);
if (det == 0) {
    printf("行列は逆行列を持ちません。¥n");
    return 1;
}

printf("¥n--- 計算結果 ---¥n");
printf("係数行列 A:¥n");
matprint(a, n, n, "%8.3f");
printf("逆行列 A^{-1}:¥n");
matprint(gyaku, n, n, "%8.3f");
printf("定数行列 b:¥n");
matprint(teisu, n, 1, "%8.3f");

// --- 解 x = A^{-1} * b の計算 ---
x = new_matrix(n, 1);
multiply(n, n, 1, gyaku, teisu, x);
printf("解 x = A^{-1} * b:¥n");
matprint(x, n, 1, "%8.3f");

```

```

// 検算結果を格納する行列を確保
b_check = new_matrix(n, 1);

// 係数行列 a と 解 x の積を計算
multiply(n, n, 1, a, x, b_check);

/* --- メモリ解放 --- */
free_matrix(a);
free_matrix(gyaku);
free_matrix(teisu);
free_matrix(x);
free_matrix(b_check);

return 0;
}

```

(イ) matinv.c

```

/*****

matinv.c -- 逆行列
*****/

#include "matutil.c" /* 行列操作用小道具集 */
#include <math.h>

double lu(int n, matrix a, int *ip) /* LU 分解 */
{
    int i, j, k, ii, ik;
    double t, u, det;
    vector weight;

    weight = new_vector(n); /* weight[0..n-1] の記憶領域確保 */
    det = 0; /* 行列式 */

```

```

for (k = 0; k < n; k++) { /* 各行について */
    ip[k] = k; /* 行交換情報の初期値 */
    u = 0; /* その行の絶対値最大の要素を求める */
    for (j = 0; j < n; j++) {
        t = fabs(a[k][j]); if (t > u) u = t;
    }
    if (u == 0) goto EXIT; /* 0 なら行列は LU 分解できない */
    weight[k] = 1 / u; /* 最大絶対値の逆数 */
}

det = 1; /* 行列式の初期値 */
for (k = 0; k < n; k++) { /* 各行について */
    u = -1;
    for (i = k; i < n; i++) { /* より下の各行について */
        ii = ip[i]; /* 重み×絶対値 が最大の行を見つける */
        t = fabs(a[ii][k]) * weight[ii];
        if (t > u) { u = t; j = i; }
    }
    ik = ip[j];
    if (j != k) {
        ip[j] = ip[k]; ip[k] = ik; /* 行番号を交換 */
        det = -det; /* 行を交換すれば行列式の符号が変わる */
    }
    u = a[ik][k]; det *= u; /* 対角成分 */
    if (u == 0) goto EXIT; /* 0 なら行列は LU 分解できない */
    for (i = k + 1; i < n; i++) { /* Gauss 消去法 */
        ii = ip[i];
        t = (a[ii][k] / u);
        for (j = k + 1; j < n; j++)
            a[ii][j] -= t * a[ik][j];
    }
}

EXIT:

free_vector(weight); /* 記憶領域を解放 */
return det; /* 戻り値は行列式 */
}

```

```

double matinv(int n, matrix a, matrix a_inv)
{
    int i, j, k, ii;

    double t, det;

    int *ip;    /* 行交換の情報 */

    ip = malloc(sizeof(int) * n);

    if (ip == NULL) error("記憶領域不足");

    det = lu(n, a, ip);

    if (det != 0)
        for (k = 0; k < n; k++) {
            for (i = 0; i < n; i++) {
                ii = ip[i];  t = (ii == k);

                for (j = 0; j < i; j++)
                    t -= a[ii][j] * a_inv[j][k];

                a_inv[i][k] = t;
            }

            for (i = n - 1; i >= 0; i--) {
                t = a_inv[i][k];  ii = ip[i];

                for (j = i + 1; j < n; j++)
                    t -= a[ii][j] * a_inv[j][k];

                a_inv[i][k] = t / a[ii][i];
            }
        }

    free(ip);

    return det;
}

/***** 以下はテスト用 *****/

#include <limits.h>

double rnd(void) /* 乱数 0 < rnd() < 1 */
{
    static unsigned long seed = 123456789UL; /* 奇数 */

```

```

    return (seed *= 69069UL) / (ULONG_MAX + 1.0);
}

int main(void)
{
    int i, j, k, n;

    matrix a, asave, a_inv;

    double s, t;

    printf("n = "); scanf("%d", &n);

    a = new_matrix(n, n); asave = new_matrix(n, n);
    a_inv = new_matrix(n, n);

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            a[i][j] = asave[i][j] = rnd() - rnd();

    printf("A\n");
    matprint(a, n, 7, "%10.6f");

    s = matinv(n, a, a_inv);
    printf("行列式 = %g\n", s);
    printf("A^{-1}\n");
    matprint(a_inv, n, 7, "%10.6f");

    t = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            s = (i == j);
            for (k = 0; k < n; k++)
                s -= asave[i][k] * a_inv[k][j];
            t += s * s;
        }

    printf("A A^{-1} の成分の二乗平均誤差 %g\n",
        sqrt(t / (n * n)));

    t = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            s = (i == j);
            for (k = 0; k < n; k++)

```



```

        s -= a_inv[i][k] * asave[k][j];

        t += s * s;
    }

    printf("A^{-1} A の成分の二乗平均誤差 %g\n",
        sqrt(t / (n * n)));

    return 0;
}

```

(ウ) matmult.c

```

/*****

matmult.c -- 行列の積

*****/

#define N 3
#define L 5
#define M 4

typedef double matNL[N][L];
typedef double matLM[L][M];
typedef double matNM[N][M];
void multiply(matNM c, matNL a, matLM b)
{
    int i, j, k;
    double s;

    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++) {
            s = 0;
            for (k = 0; k < L; k++) s += a[i][k] * b[k][j];
            c[i][j] = s;
        }
}

typedef double matNN[N][N];

```

```

void mult2(matNN a, matNN b)
{
    int i, j, k;
    double s, temp[N];

    for (i = 0; i < N; i++) {
        for (k = 0; k < N; k++) temp[k] = a[i][k];
        for (j = 0; j < N; j++) {
            s = 0;
            for (k = 0; k < N; k++) s += temp[k] * b[k][j];
            a[i][j] = s;
        }
    }
}

```

```

#include <stdio.h>

```

```

#include <stdlib.h>

```

```

void matprint(int nrow, int ncol, double *a)
{
    int i, j;

    for (i = 0; i < nrow; i++) {
        for (j = 0; j < ncol; j++) printf("%8.1f", *a++);
        printf("\n");
    }
}

```

```

int main(void)
{
    int i, j;
    static matNL a;
    static matLM b;
    static matNM c;
    static matNN d, e;
}

```

```

    for (i = 0; i < N; i++)
        for (j = 0; j < L; j++)
            a[i][j] = rand() / (RAND_MAX / 10 + 1);
    for (i = 0; i < L; i++)
        for (j = 0; j < M; j++)
            b[i][j] = rand() / (RAND_MAX / 10 + 1);
    printf("A\n");  matprint(N, L, (double *)a);
    printf("B\n");  matprint(L, M, (double *)b);
    multiply(c, a, b);
    printf("AB\n");  matprint(N, M, (double *)c);

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++) {
            d[i][j] = rand() / (RAND_MAX / 10 + 1);
            e[i][j] = rand() / (RAND_MAX / 10 + 1);
        }
    printf("D\n");  matprint(N, N, (double *)d);
    printf("E\n");  matprint(N, N, (double *)e);
    mult2(d, e);
    printf("DE\n");  matprint(N, N, (double *)d);

    return 0;
}

```

(工) matutil.c

```

/*****

    matutil.c -- 行列
    *****/

/* 行列操作の小道具集 */

#ifndef MATUTIL
#define MATUTIL

```

```

#include <stdio.h>
#include <stdlib.h>
#ifndef SCALAR
    #define SCALAR double
#endif
typedef SCALAR *vector, **matrix;

void error(char *message)
{
    fprintf(stderr, "%n%s%n", message);  exit(1);
}

vector newvec(int n)
{
    return malloc(sizeof(SCALAR) * n);
}

matrix newmat(int nrow, int ncol)
{
    int i;
    matrix a;

    a = malloc((nrow + 1) * sizeof(void *));
    if (a == NULL) return NULL;  /* 記憶領域不足 */
    for (i = 0; i < nrow; i++) {
        a[i] = malloc(sizeof(SCALAR) * ncol);
        if (a[i] == NULL) {
            while (--i >= 0) free(a[i]);
            free(a);  return NULL;  /* 記憶領域不足 */
        }
    }
    a[nrow] = NULL;  /* 行の数を自動判断するための工夫 */
    return a;
}

vector new_vector(int n)

```

```

{
    vector v;

    v = newvec(n);
    if (v == NULL) error("記憶領域不足.");
    return v;
}

matrix new_matrix(int nrow, int ncol)
{
    matrix a;

    a = newmat(nrow, ncol);
    if (a == NULL) error("記憶領域不足.");
    return a;
}

void free_vector(vector v)
{
    free(v);
}

void free_matrix(matrix a)
{
    matrix b;

    b = a;
    while (*b != NULL) free(*b++);
    free(a);
}

double innerproduct(int n, vector u, vector v)
{
    int i, n5;
    double s;

```

```

    s = 0;    n5 = n % 5;
    for (i = 0; i < n5; i++) s += u[i]*v[i];
    for (i = n5; i < n; i += 5)
        s += u[i]*v[i] + u[i+1]*v[i+1] + u[i+2]*v[i+2]
            + u[i+3]*v[i+3] + u[i+4]*v[i+4];

    return s;
}

void vecprint(vector v, int n, int perline, char *format)
{
    int j, k;

    k = 0;
    for (j = 0; j < n; j++) {
        printf(format, v[j]);
        if (++k >= perline) {    k = 0;    printf("\n");    }
    }
    if (k != 0) printf("\n");
}

void matprint(matrix a, int ncol, int perline, char *format)
{
    int i;

    for (i = 0; a[i] != NULL; i++) {
        vecprint(a[i], ncol, perline, format);
        if (ncol > perline) printf("\n");
    }
}

void multiply(int m, int k, int p, matrix a, matrix b, matrix c) {
    int i, j, l;
    for (i = 0; i < m; i++) {
        for (j = 0; j < p; j++) {
            c[i][j] = 0.0;
            for (l = 0; l < k; l++) {

```

```

        c[i][j] += a[i][i] * b[i][j];
    }
}

}

}

}

#endif

/* ***** 以下は簡単な使用例 ***** */
#include <time.h>
int main(void)
{
    int i, j, nrow, ncol;
    matrix a;

    srand((unsigned int) time(NULL)); /* 乱数の種を設定 */
    nrow = rand() % 10 + 1;           /* 行の数 */
    ncol = rand() % 10 + 1;           /* 列の数 */
    a = new_matrix(nrow, ncol);       /* 行列を作る */
    for (i = 0; i < nrow; i++)        /* 乱数を代入 */
        for (j = 0; j < ncol; j++)
            a[i][j] = 10.0 * rand() / RAND_MAX;
    printf("%d x %d 行列:\n", nrow, ncol);
    matprint(a, ncol, 4, " % -14g"); /* 出力 */
    return 0;
}

#endif
#endif /* 最初の #ifndef ... に対応する */

```

2. 実行結果

行列の次元 $n = 3$ 係数行列 A ($n \times n$) を入力してください: $a[0][0] = 2$ $a[0][1] = 1$ $a[0][2] = 1$ $a[1][0] = 1$ $a[1][1] = 2$ $a[1][2] = 1$ $a[2][0] = 1$ $a[2][1] = 1$ $a[2][2] = 2$ 定数行列 b ($n \times 1$) を入力してください: $b[0][0] = 1$ $b[1][0] = 2$ $b[2][0] = 3$
 --- 計算結果 --- 係数行列 A : 2.000 1.000 1.000 1.000 2.000 1.000 1.000 1.000 2.000 逆行列 A^{-1} : 0.750 -0.250 -0.250 -0.250 0.750 -0.250 -0.250 -0.250 0.750 定数行列 b : 1.000 2.000 3.000 解 $x = A^{-1} * b$: -0.500 0.500 1.500